

System Design Write-Up — Rent & Flatmate Finder

Architecture Overview

The system is a FastAPI (Python) backend backed by MongoDB via the Beanie ODM, paired with a React/Vite frontend. A single ASGI app hosts both the REST API and a Socket.IO server (`socket_app = socketio.ASGIApp(sio, other_asgi_app=app)`), so one process on Render serves everything. The frontend deploys separately to Vercel and talks to the backend over HTTPS (REST) and WSS (chat).

Compatibility Scoring Design

Every tenant–listing pair gets **one** compatibility score, computed lazily the first time it's needed (when a tenant expresses interest, or when listings are ranked) and cached permanently in a dedicated `compatibility_scores` collection, keyed by a unique (`tenant_id`, `listing_id`) index. Subsequent lookups read the cached row instead of re-invoking the LLM, satisfying the "not recomputed on every request" requirement while keeping the read path cheap.

The scoring function (`llm_service.get_compatibility_score`) is provider-agnostic: `LLM_PROVIDER` selects between **Groq** (OpenAI-compatible completions, used in production) and a local **Ollama** model (offline dev). The prompt (`build_prompt.py`) sends the tenant's cleaned profile and listing fields, and instructs the model to score against an explicit rubric — 50 points for location match, 50 for budget fit — returning strict JSON (`{score, explanation}`), no markdown. A Pydantic model validates the shape, clamps the score into `[0, 100]`, and guarantees a non-empty explanation; on the first parse failure it retries once, and on a second failure or any transport error it falls back to `rule_based_score()`.

The rule-based fallback mirrors the same rubric deterministically: exact/partial word-overlap on location for up to 50 points, and a graduated budget check (full 50 points inside the tenant's range, smaller partial credit the further rent drifts outside it) for the other 50, with a human-readable explanation assembled from the same signals. Because both paths are guaranteed to return a value, `get_compatibility_score()` never raises — it always yields `{score, explanation, method}`, where `method` (`llm-groq`, `llm-ollama`, or `rule-based`) is persisted alongside the score so the scoring source stays auditable. This means listing scoring degrades gracefully rather than blocking the interest/browse flow if the LLM provider is down or misconfigured.

LLM Integration and Fallback

Provider selection and credentials live entirely in environment variables (`LLM_PROVIDER`, `GROQ_API_KEY/GROQ_MODEL`, or `LLM_BASE_URL/LLM_MODEL` for Ollama), so switching providers is a config change, not a code change. Both calls run through `httpx.AsyncClient` with explicit timeouts (15s Ollama, 30s Groq) so a hung LLM can't stall a request — a timeout is caught by the same handler that triggers the rule-based fallback. This layered design (validate → retry once → fall back) was chosen over failing outright, since scoring is a ranking signal, not a gate on whether a tenant can express interest.

Chat Implementation

Chat is scoped to an **accepted interest**, not to a user pair directly — the room name is `room:{interest_id}`. On `connect`, the Socket.IO server verifies the JWT passed in the `auth` payload, rejects the handshake if it's missing/invalid, and auto-joins the socket to a personal room `user:{user_id}` (used for

future targeted server-push events). To enter a chat room the client emits `join_room` with an `interest_id`; the server re-validates that the interest exists, is `accepted` (or `revoked`, in which case the room opens read-only for history), and that the connecting user is either the tenant or the listing's owner — preventing unauthorized users from joining someone else's conversation even if they guess an ID. `send_message` re-runs the same participant/status checks before persisting the message via Beanie (`Message` document, indexed on `(interest_id, created_at)` for fast pagination) and broadcasting `new_message` to everyone in the room. If an owner later revokes access, sends are blocked server-side while history remains readable — enforced at the socket layer, not just the UI.

Notification Flow

Two email triggers fire from the interest lifecycle, both via `fastapi-mail` over SMTP and both wrapped so a delivery failure is logged, not raised — an SMTP outage can never break the interest/accept flow:

1. **High-match alert to owner** — when a tenant creates an interest, the backend computes (or fetches the cached) compatibility score; if `score >= 80`, `notify_owner_high_score_interest` emails the owner immediately, before the owner has even opened the app.
2. **Response notification to tenant** — when the owner calls `PATCH /interests/{id}/respond`, `notify_tenant_interest_response` emails the tenant whether they were accepted or declined, with an accept email nudging them toward the new chat room.

Data Modeling

Six core Beanie documents — `User`, `Listing`, `TenantProfile`, `Interest`, `Message`, `CompatibilityScore` — keep relationships as plain string ID references (no embedding), which keeps each collection independently indexable and avoids document-size growth on high-traffic listings. Uniqueness constraints (`email` on users, `(tenant_id, listing_id)` on both interests and scores) are enforced at the database level via compound indexes rather than only in application code.